

MODULE 17

ANN Project Implementation

Topics Covered

Lectures 1–8 — End-to-End ANN Classification Project with Deployment
Lecture 9 — Regression Problem using ANN
Lectures 10–13 — Hyperparameter Tuning with KerasTuner

Lectures 1–8: End-to-End ANN Classification Project

This project builds a customer churn prediction model using the Churn Modelling dataset (bank customer data). The full pipeline: load → clean → encode → scale → build ANN → predict → deploy as a Streamlit web app.

Phase	Lecture Key Task
Project setup	L1 — Load dataset, explore, define objective
Data preprocessing	L2 — Handle missing values, drop irrelevant columns
Encoding & split	L3 — Label encode, one-hot encode, scale, train-test split
Build ANN	L4 — Define model architecture, compile, train
Predict	L5 — Make predictions, threshold tuning, evaluate
Save & load	L6 — Pickle encoder & model, load for inference
Streamlit UI	L7 — Build interactive web app
Deploy	L8 — Push to GitHub, deploy on cloud

L1–L2: Data Loading & Cleaning

```
import pandas as pd
import numpy as np

df = pd.read_csv('Churn_Modelling.csv')
print(df.shape)          # (10000, 14)
print(df.head())
print(df.isnull().sum()) # check missing values

# Drop irrelevant columns
df.drop(columns=['RowNumber', 'CustomerId', 'Surname'], inplace=True)

# Target distribution
print(df['Exited'].value_counts())
# 0 (stayed): 7963    1 (churned): 2037 — imbalanced dataset

# Feature overview
print(df.describe())
print(df.dtypes)
```

L3: Feature Encoding, Scaling & Train-Test Split

```
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
import pickle

# Label encode binary categorical: Gender (Male/Female → 0/1)
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])
pickle.dump(le, open('label_encoder_gender.pkl', 'wb')) # save for inference

# One-Hot encode Geography (France/Germany/Spain)
ohe = OneHotEncoder(sparse_output=False)
```

```

geo_encoded = ohe.fit_transform(df[['Geography']])
geo_df      = pd.DataFrame(geo_encoded,
                           columns=ohe.get_feature_names_out(['Geography']))
df = pd.concat([df.drop('Geography', axis=1), geo_df], axis=1)
pickle.dump(ohe, open('ohe_geography.pkl', 'wb')) # save for inference

# Features and target
X = df.drop('Exited', axis=1)
y = df['Exited']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Scale numerical features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
pickle.dump(scaler, open('scaler.pkl', 'wb')) # save for inference

```

L4: Building & Training the ANN

```

import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),

    Dense(32, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),

    Dense(1, activation='sigmoid') # binary classification output
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()

callbacks = [
    EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True),
    ModelCheckpoint('best_model.h5', save_best_only=True)
]

history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=callbacks
)

```

L5: Prediction & Evaluation

```
from sklearn.metrics import (classification_report, confusion_matrix,
                             accuracy_score, roc_auc_score)
import matplotlib.pyplot as plt

# Predict probabilities
y_prob = model.predict(X_test).flatten()

# Default threshold = 0.5
y_pred = (y_prob >= 0.5).astype(int)

print('Accuracy: ', accuracy_score(y_test, y_pred))
print('ROC-AUC: ', roc_auc_score(y_test, y_prob))
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))

# Plot training history
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.plot(history.history['loss'], label='Train Loss')
ax1.plot(history.history['val_loss'], label='Val Loss')
ax1.legend(); ax1.set_title('Loss Curve')

ax2.plot(history.history['accuracy'], label='Train Acc')
ax2.plot(history.history['val_accuracy'], label='Val Acc')
ax2.legend(); ax2.set_title('Accuracy Curve')
plt.tight_layout(); plt.show()
```

L6: Saving & Loading Model + Encoders

```
import pickle
import tensorflow as tf

# Save model
model.save('churn_model.h5')

# Load everything for inference
model = tf.keras.models.load_model('churn_model.h5')
scaler = pickle.load(open('scaler.pkl', 'rb'))
le = pickle.load(open('label_encoder_gender.pkl', 'rb'))
ohe = pickle.load(open('ohe_geography.pkl', 'rb'))

# Predict for a single new customer
import pandas as pd

new_customer = pd.DataFrame([{'CreditScore': 600, 'Gender': 'Male', 'Age': 40,
                              'Tenure': 3, 'Balance': 60000, 'NumOfProducts': 2,
                              'HasCrCard': 1, 'IsActiveMember': 1,
                              'EstimatedSalary': 50000, 'Geography': 'France'}])

new_customer['Gender'] = le.transform(new_customer['Gender'])
geo_enc = ohe.transform(new_customer[['Geography']])
geo_df = pd.DataFrame(geo_enc, columns=ohe.get_feature_names_out(['Geography']))
new_customer = pd.concat([new_customer.drop('Geography', axis=1), geo_df], axis=1)
new_customer_scaled = scaler.transform(new_customer)

prob = model.predict(new_customer_scaled)[0][0]
print(f'Churn probability: {prob:.2%}') # e.g. 18.34%
print('Churn prediction:', 'YES' if prob >= 0.5 else 'NO')
```

L7: Streamlit Web App

```
# app.py
import streamlit as st
import pandas as pd, numpy as np, pickle, tensorflow as tf

model = tf.keras.models.load_model('churn_model.h5')
scaler = pickle.load(open('scaler.pkl', 'rb'))
le = pickle.load(open('label_encoder_gender.pkl', 'rb'))
ohe = pickle.load(open('ohe_geography.pkl', 'rb'))

st.title('Customer Churn Prediction')
st.markdown('Enter customer details to predict churn probability.')

# Input widgets
geography = st.selectbox('Geography', ohe.categories_[0])
gender = st.selectbox('Gender', le.classes_)
age = st.slider('Age', 18, 92, 40)
credit_score = st.number_input('Credit Score', 300, 900, 600)
balance = st.number_input('Balance', 0.0, 250000.0, 60000.0)
salary = st.number_input('Estimated Salary', 0.0, 200000.0, 50000.0)
tenure = st.slider('Tenure (years)', 0, 10, 3)
products = st.selectbox('Number of Products', [1, 2, 3, 4])
has_card = st.selectbox('Has Credit Card', [0, 1])
is_active = st.selectbox('Is Active Member', [0, 1])

if st.button('Predict'):
    # Assemble and preprocess
    row = pd.DataFrame([{'CreditScore': credit_score, 'Gender': le.transform([gender])[0],
                        'Age': age, 'Tenure': tenure, 'Balance': balance,
                        'NumOfProducts': products, 'HasCrCard': has_card,
                        'IsActiveMember': is_active, 'EstimatedSalary': salary
                        }])
    geo_enc = ohe.transform([[geography]])
    geo_df = pd.DataFrame(geo_enc,
                          columns=ohe.get_feature_names_out(['Geography']))
    row = pd.concat([row, geo_df], axis=1)
    row_scaled = scaler.transform(row)
    prob = model.predict(row_scaled)[0][0]
    st.metric('Churn Probability', f'{prob:.2%}')
    if prob >= 0.5:
        st.error('❌ High churn risk!')
    else:
        st.success('✅ Customer likely to stay.')
```

L8: Deployment

Step	Action
1. requirements.txt	streamlit, tensorflow, scikit-learn, pandas, numpy
2. Push to GitHub	git init → git add . → git commit -m 'churn app' → git push
3. Streamlit Cloud	share.streamlit.io → New app → Connect GitHub repo → Deploy
4. Render (alt)	New Web Service → repo → start command: streamlit run app.py --server.port \$PORT
5. Environment vars	Set secrets (API keys) in platform dashboard, not in code

Lecture 9: Regression Problem using ANN

The same ANN architecture applies to regression — predict a continuous value (e.g. house price) instead of a class. The key differences are the output layer and loss function.

Aspect	Classification vs Regression
Output layer	Dense(1, activation='sigmoid') or Dense(n, activation='softmax') vs Dense(1) — no activation
Loss function	binary_crossentropy / categorical_crossentropy vs mse or mae
Metrics	accuracy, AUC vs mae, mse, r2_score
Prediction	Probability → threshold → class vs Continuous number directly

```
# Example: House Price Prediction
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, r2_score
import numpy as np

data = fetch_california_housing()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Regression ANN
model = tf.keras.Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(16, activation='relu'),
    Dense(1) # NO activation — raw continuous output
])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])

history = model.fit(X_train, y_train, epochs=100, batch_size=32,
                    validation_split=0.2,
                    callbacks=[EarlyStopping(patience=10,
                                             restore_best_weights=True)])

y_pred = model.predict(X_test).flatten()
print(f'MAE: {mean_absolute_error(y_test, y_pred):.4f}')
print(f'RMSE: {np.sqrt(((y_test - y_pred)**2).mean()):.4f}')
print(f'R²: {r2_score(y_test, y_pred):.4f}')
```

Lectures 10–13: Hyperparameter Tuning with KerasTuner

What is Hyperparameter Tuning?

Hyperparameters are settings you choose before training — number of layers, neurons per layer, learning rate, dropout rate, optimiser. Tuning finds the combination that gives the best validation performance automatically.

Hyperparameter	What It Controls
Number of hidden layers	Model depth — capacity to learn complex patterns
Neurons per layer	Width — how many features each layer can represent
Learning rate	Step size for gradient descent
Dropout rate	Regularisation strength
Batch size	Gradient noise and training stability
Optimiser	Which optimiser algorithm to use
Activation function	Non-linearity type per layer

KerasTuner — Setup & Search Strategies

```
pip install keras-tuner

import keras_tuner as kt

# KerasTuner search strategies:
# RandomSearch - randomly sample from search space (fast, simple)
# Hyperband    - early stopping of bad trials (efficient)
# BayesianOpt   - uses past results to intelligently pick next trial (best)
```

L10: Tuning Learning Rate & Neurons

```
def build_model(hp):
    model = tf.keras.Sequential()

    # Tune number of neurons in first hidden layer
    model.add(Dense(
        units=hp.Int('units_1', min_value=32, max_value=256, step=32),
        activation='relu',
        input_shape=(X_train.shape[1],)
    ))
    model.add(Dropout(
        rate=hp.Float('dropout_1', min_value=0.1, max_value=0.5, step=0.1)
    ))

    model.add(Dense(
        units=hp.Int('units_2', min_value=16, max_value=128, step=16),
        activation='relu'
    ))

    model.add(Dense(1, activation='sigmoid'))

    # Tune learning rate
    lr = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
                  loss='binary_crossentropy', metrics=['accuracy'])
    return model

tuner = kt.RandomSearch(
    build_model,
```

```

    objective='val_accuracy',
    max_trials=20,
    directory='kt_results',
    project_name='churn_tuning'
)

tuner.search(X_train, y_train, epochs=30,
             validation_split=0.2,
             callbacks=[EarlyStopping(patience=5)])

best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]
print('Best units_1: ', best_hp.get('units_1'))
print('Best units_2: ', best_hp.get('units_2'))
print('Best lr:      ', best_hp.get('learning_rate'))
print('Best dropout: ', best_hp.get('dropout_1'))

```

L11: Tuning Number of Nodes per Layer

```

def build_model_nodes(hp):
    model = tf.keras.Sequential()

    # Tune 3 layers independently
    for i in range(1, 4):
        model.add(Dense(
            units=hp.Int(f'layer_{i}_units',
                        min_value=16, max_value=256, step=16),
            activation=hp.Choice(f'activation_{i}',
                                values=['relu', 'tanh', 'elu']),
        ))
        model.add(BatchNormalization())
        model.add(Dropout(
            rate=hp.Float(f'dropout_{i}', 0.1, 0.5, step=0.1)
        ))

    model.add(Dense(1, activation='sigmoid'))
    model.compile(optimizer='adam', loss='binary_crossentropy',
                  metrics=['accuracy'])
    return model

tuner = kt.Hyperband(
    build_model_nodes,
    objective='val_accuracy',
    max_epochs=50,
    factor=3,
    directory='kt_hyperband',
    project_name='nodes_tuning'
)

```

L12: Tuning Number of Hidden Layers

```

def build_model_layers(hp):
    model = tf.keras.Sequential()
    model.add(Dense(64, activation='relu',
                    input_shape=(X_train.shape[1],)))

    # Tune how many ADDITIONAL hidden layers to add (0 to 4)
    n_layers = hp.Int('n_hidden_layers', min_value=0, max_value=4)
    for i in range(n_layers):
        model.add(Dense(
            units=hp.Int(f'units_{i}', 16, 128, step=16),

```

```

        activation='relu'
    ))
    model.add(Dropout(0.3))

    model.add(Dense(1, activation='sigmoid'))
    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            hp.Choice('lr', [1e-2, 1e-3, 1e-4])),
        loss='binary_crossentropy', metrics=['accuracy']
    )
    return model

tuner = kt.BayesianOptimization(
    build_model_layers,
    objective='val_accuracy',
    max_trials=30,
    directory='kt_bayesian',
    project_name='layers_tuning'
)

```

L13: Full Tuning — Layers + Nodes + Optimiser

```

def build_full_model(hp):
    model = tf.keras.Sequential()

    # Input + first layer
    model.add(Dense(
        units=hp.Int('input_units', 32, 256, step=32),
        activation='relu',
        input_shape=(X_train.shape[1],)))
    model.add(BatchNormalization())

    # Variable number of hidden layers
    for i in range(hp.Int('n_layers', 1, 4)):
        model.add(Dense(
            units=hp.Int(f'units_{i}', 16, 128, step=16),
            activation=hp.Choice('activation', ['relu', 'elu', 'selu'])))
        model.add(Dropout(
            hp.Float(f'drop_{i}', 0.1, 0.5, step=0.1)))

    model.add(Dense(1, activation='sigmoid'))

    # Tune optimiser
    optimizer_choice = hp.Choice('optimizer', ['adam', 'rmsprop', 'sgd'])
    lr = hp.Float('lr', 1e-4, 1e-2, sampling='log')

    if optimizer_choice == 'adam':
        opt = tf.keras.optimizers.Adam(lr)
    elif optimizer_choice == 'rmsprop':
        opt = tf.keras.optimizers.RMSprop(lr)
    else:
        opt = tf.keras.optimizers.SGD(lr, momentum=0.9)

    model.compile(optimizer=opt, loss='binary_crossentropy',
                  metrics=['accuracy'])
    return model

tuner = kt.BayesianOptimization(
    build_full_model,
    objective='val_accuracy',
    max_trials=50,

```

```

    directory='kt_full',
    project_name='full_tuning'
)

tuner.search(X_train, y_train, epochs=50,
             validation_split=0.2,
             callbacks=[EarlyStopping(patience=5)])

# Get best model - already compiled with best hyperparameters
best_model = tuner.get_best_models(num_models=1)[0]

# Retrain best model on full training data
best_hp = tuner.get_best_hyperparameters(1)[0]
final_model = tuner.hypermodel.build(best_hp)
final_model.fit(X_train, y_train, epochs=100,
                callbacks=[EarlyStopping(patience=10,
                                         restore_best_weights=True)])

# Evaluate
loss, acc = final_model.evaluate(X_test, y_test)
print(f'Final Test Accuracy: {acc:.4f}')

# Summary of best hyperparameters
tuner.results_summary(num_trials=5)

```

KerasTuner Search Space Reference

hp Method	Usage Example
hp.Int(name, min, max, step)	Integer range → hp.Int('units', 32, 256, step=32)
hp.Float(name, min, max, step)	Float range → hp.Float('dropout', 0.1, 0.5, step=0.1)
hp.Float(..., sampling='log')	Log scale range → learning rate: hp.Float('lr', 1e-4, 1e-2, sampling='log')
hp.Choice(name, values)	Pick from list → hp.Choice('optimizer', ['adam','sgd'])
hp.Boolean(name)	True/False toggle → hp.Boolean('use_batchnorm')

Tuner Strategy	How It Works Best For
RandomSearch	Randomly samples from search space. Fast but no intelligence. Good for initial exploration.
Hyperband	Runs many trials briefly, kills bad ones early, trains good ones longer. Efficient on time.
BayesianOptimization	Builds probabilistic model of search space. Picks next trial smartly. Best for limited compute.

End of Module 17 Notes — Generative AI Course